

Endurance Racing RL Project Blueprint

1. Executive Summary & Core Philosophy

Project Vision

Build an interactive, web-based race-strategy sandbox that showcases end-to-end expertise across data analytics, reinforcement learning (RL) math, motorsport engineering, and intuitive UI design.

The application enables an employer or reviewer to adjust real-world strategic variables (caution frequency, tyre degradation rates, traffic density, fuel burn), click "Run Strategy," and visually evaluate how a trained Reinforcement Learning agent adapts compared to traditional human rule-based strategies.

Technical Stack

- **Language & Core Environment:** Python 3.11+, Gymnasium
- **Data Processing:** DuckDB, Pandas, NumPy
- **RL Framework:** Stable-Baselines3 (or custom PyTorch)
- **Frontend & Interactive UI:** Streamlit, Plotly
- **Model Export & Deployment:** ONNX / Client-side inference via PyTorch/ONNX Runtime

Governing Architecture Principle: Strict Separation of Concerns

To prevent scope creep and maintain software modularity, the project strictly isolates components:

- The Physics & MDP Engine (`gymnasium.Env`) does not import UI libraries and has no awareness of Streamlit.
- The RL Agent strictly interacts with the standard Gym API (`step`, `reset`, `action_space`, `observation_space`).
- The Streamlit UI handles rendering and parameter pass-through; it never contains direct environment step physics or neural network training loops.

2. Motorsport Domain & Interactive RL Foundations

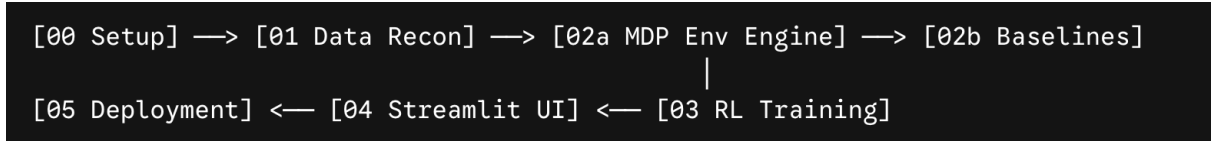
Motorsport Strategy Engine: Rules Boundary

<i>Strategic Dimension</i>	<i>IMSA (e.g., Daytona 24h)</i>	<i>WEC (e.g., Le Mans 24h)</i>
Primary Safety Mechanism	Full-Course Yellow (FCY) + Safety Car + Pass-Around / Wave-Around procedures.	FCY (80 km/h limiter), Virtual Safety Car (VSC), and Safety Car (SC).
Pit Lane Rule under Caution	Pit lane closes immediately upon FCY call; opens sequentially (Prototypes first, GTs second).	Pit lane remains open under FCY / VSC (subject to local yellow rules).
Strategic Opportunity	High volatility. Pitting right before an FCY grants a massive time delta ("cheap pit stop").	Moderate volatility. FCY/VSC reduces pit loss time delta relative to green flag speeds.
Traffic Dynamics	High traffic density; frequent multi-class overtaking maneuvers.	Track dependent; Le Mans features long straights with lower traffic friction per lap.

Interactive RL Principles

1. **Action Advice & Policy Override:** The system allows human operators to act as supervisors, offering manual overrides during critical events (e.g., Caution Called) to compare human intuition against policy outputs.
2. **Agent Transparency & Explainability (XAI):** Rather than treating the policy as a black box, the dashboard visualizes underlying action value estimates $Q(s,a)$ or action probabilities $P(a|s)$ to explain why an action was chosen.
3. **Cognitive Ergonomics:** High-speed stepping runs automatically (e.g., 5 laps/sec), but automatically pauses on key strategic triggers (Caution Called, Rain Started, Fuel Critical) to minimize decision fatigue.

3. High-Level Stage Roadmap



Stage Summary Matrix

Stage	Focus Area	Primary Deliverable	Boundary Constraint
00	Infrastructure	Virtualenv & Repo Structure	No data analysis or simulation code.
01	Data Recon	DuckDB CSV / Parquet extraction	Extract constants only; no environment code.
02a	Core MDP Engine	Pure gymnasium.Env Python class	Zero UI imports; zero race control/wave-around logic.
02b	Rule Baselines	Hardcoded strategy classes	Pure algorithmic rules based on current state.
03	RL Agent Training	Saved Policy Model (.zip / .onnx)	Offline execution; output saved checkpoints only.
04	Streamlit UI	Interactive Multi-Page App	UI & state presentation layer only.
05	Portfolio Packaging	Live Hosted App + Documentation	Final polish and public cloud deployment.

4. Stage Specifications

Stage 00: Infrastructure & Environment Setup

- Status: COMPLETE
- Deliverables: Python virtual environment, directory layout (/data, /envs, /agents, /ui, /tests), standard project dependencies.

Stage 01: Data Reconnaissance & Parameter Extraction

- Status: COMPLETE
- Deliverables: Extracted constants derived from empirical dataset (~1.6M laps via DuckDB):
 - Baseline green-flag lap time (T_{base})
 - Base fuel burn rate (Δf_{base} per lap)
 - Non-linear tyre degradation constants (α deg, β deg)
 - Pit stop duration time penalties (T_{pit_fuel} , T_{pit_tyres} , T_{pit_full})
 - Stochastic event rates: Per-lap caution probability (PFCY)

Stage 02a: Core MDP Simulation Engine (gymnasium.Env)

- Status: IN PROGRESS
- Core Paradigm: Cautions are modeled strictly as Markov state modifiers, not as complex procedural race-control simulations.

Caution Transition Mechanics

$$\text{If Flag}_t = \text{Caution} : \begin{cases} T_{\text{pit}} \leftarrow T_{\text{pit}} \times \gamma_{\text{pit_fcy}} & (\gamma \approx 0.50 - 0.65) \\ \Delta f \leftarrow \Delta f \times \gamma_{\text{fuel_fcy}} & (\gamma \approx 0.30) \\ \Delta d \leftarrow \Delta d \times \gamma_{\text{deg_fcy}} & (\gamma \approx 0.20) \end{cases}$$

Observation Space (S)

Feature	Raw Unit Range	Normalization Formula
Lap Progress	$1 \dots N_{\text{laps}}$	t / N_{laps}
Fuel Remaining	$0.0 \dots 1.0$ (ratio)	f_t
Tyre Wear	$0.0 \dots 1.0$ (0 = Fresh, 1 = Worn)	d_t
Gap to Leader	$0.0 \dots 120.0$ seconds	$\Delta_{\text{lead}} / 120.0$
Gap to Behind	$0.0 \dots 120.0$ seconds	$\Delta_{\text{behind}} / 120.0$
Flag State	0 = Green, 1 = Caution	Binary
Stint Age	$0 \dots 40$ laps	$t_{\text{stint}} / 40.0$

Action Space (A)

Discrete action space containing 5 decisions:

- 0: Push Pace ($\rightarrow -0.5s$ lap time penalty, +20% fuel burn, +25% tyre wear)
- 1: Normal Pace ($\rightarrow$ Standard empirical parameters)
- 2: Fuel Save ($\rightarrow +0.8s$ lap time penalty, -25% fuel burn, -10% tyre wear)
- 3: Box (Fuel Only) ($\rightarrow$ Incurs T_{pit_fuel} , resets fuel $f_t \rightarrow 1.0$)
- 4: Box (Fuel + Tyres) ($\rightarrow$ Incurs T_{pit_full} , resets $f_t \rightarrow 1.0$ and $dt \rightarrow 0.0$)

Reward Function (R)

$$r_t = - (t_{lap}(s_t, a_t) + \text{PitPenalty}(a_t)) - \mathbb{I}_{DNF} \cdot 1000$$

Execution Steps for `env.step(action)`

Python

```
# 1. Obtain Pace Modifiers
lap_time_delta, fuel_mult, wear_mult = self._get_action_modifiers(action)

# 2. Transition Flag State (Stochastic Transition)
is_caution = self.np_random.random() < self.p_caution_per_lap

# 3. Compute Lap Dynamics & Consumables
if is_caution:
    lap_time = self.t_base_fcy
    fuel_burned = self.base_fuel_burn * 0.3
    tyre_wear_added = self.base_wear_rate * 0.2
else:
    tyre_deg_penalty = self.alpha_deg * (self.tyre_wear ** 2)
    lap_time = self.t_base_green + lap_time_delta + tyre_deg_penalty + self._get_tra
    fuel_burned = self.base_fuel_burn * fuel_mult
    tyre_wear_added = self.base_wear_rate * wear_mult

# 4. Handle Pit Stop Execution
if action in [3, 4]:
    pit_loss = self.t_pit_fcy if is_caution else self.t_pit_green
    if action == 4:
        pit_loss += self.t_tyre_change_penalty
        self.tyre_wear = 0.0
    lap_time += pit_loss
    self.fuel = 1.0

# 5. State Update & Reward Computation
self.fuel = max(0.0, self.fuel - fuel_burned)
self.tyre_wear = min(1.0, self.tyre_wear + tyre_wear_added)

reward = -lap_time
terminated = False
if self.fuel <= 0.0:
    reward -= 1000.0 # Out of fuel penalty
    terminated = True
```

Stage 02a Verification Gate

Stage 02a is complete only when:

1. `grep -r "streamlit" envs/` returns zero results.
2. The environment file `endurance_env.py` remains under 250 lines of code.
3. A headless test script executes 1,000 random steps without error:

```
import gymnasium as gym
from envs.endurance_env import EnduranceRaceEnv

env = EnduranceRaceEnv()
obs, info = env.reset()
done = False
total_reward = 0

while not done:
    action = env.action_space.sample()
    obs, reward, terminated, truncated, info = env.step(action)
    total_reward += reward
    done = terminated or truncated

print(f"Test Passed. Total Elapsed Time Penalty: {total_reward:.2f}s")
```

Stage 02b: Heuristic Baselines & Benchmarks

- Status: PLANNED
- Scope: Build rule-based strategy classes to evaluate against the RL agent.
- Deliverables:
 1. FixedWindowStrategy: Pits at fixed lap intervals or when fuel drops below 5%.
 2. ReactiveCautionStrategy: Pits immediately when a caution occurs if fuel is below 50%.

Stage 03: Offline RL Agent Training

- Status: PLANNED
- Scope: Train a Deep Q-Network (DQN) or Proximal Policy Optimization (PPO) agent on the Stage 02a Gym environment.
- Deliverables:
 - `train.py` training pipeline using Stable-Baselines3.
 - Exported model checkpoint (`endurance_agent.zip` or `.onnx`).
 - Evaluation script comparing RL Agent completion times against Stage 02b heuristic baselines.

Stage 04: Interactive Streamlit UI

- Status: PLANNED
- Scope: Develop the interactive demonstration interface.
- Deliverables:
 - Sidebar Parameter Controls: Sliders for caution likelihood, tyre degradation multipliers, and fuel burn rates.

- Interactive Step Engine: Lap-by-lap simulation controls with manual override options.
- Explainability Dashboard: Real-time rendering of action values $Q(s,a)$, fuel telemetry, tyre health curves, and gap charts relative to the baseline.

Stage 05: Portfolio Packaging & Deployment

- Status: PLANNED
- Scope: Final documentation, repository clean-up, and public cloud deployment.
- Deliverables:
 - Cloud deployment on Streamlit Community Cloud or AWS/GCP.
 - Architectural documentation and system diagrams in the main README.md.
 - Summary portfolio narrative highlighting problem formulation, data integration, and strategic performance gains.